HEXAWARE

Capstone 2-
Build & Ship an Agentic Solution

June 2026

HEXAWARE

TRACK A · AI IN SDLC

Agentic Coding inside VS Code + Copilot

Turn the IDE into an autonomous engineering environment. Extend GitHub Copilot with custom agents, hooks, skills and prompts so it can comprehend, build and fix code with minimal hand-holding — while you stay in control.

Custom agents

Task-scoped agents (comprehend / build / fix) with their own instructions & tools.

Hooks

Fire on events — pre-commit, on-save, on-PR — to run checks or trigger an agent.

Skills

Reusable capabilities the agent can call: run tests, search repo, scaffold, lint.

Prompts

Curated, versioned prompt templates encoding standards and review rules.

Toolbox: VS Code · GitHub Copilot (agent mode + custom chat modes) · MCP servers for repo, tests & CI · *the agent acts; the engineer reviews & approves.*

Three Autonomy Challenges

Choose one scenario as your deliverable. Each pushes the agent further toward autonomy; all three require human approval before code is merged.

Code Comprehension

Point the agent at an unfamiliar repo. It maps complete architecture, explains key flows with citations, creates a visual knowledge graph and produces an onboarding doc + change-impact analysis based on user queries

Cited explanation + module map + impact set

Spec → Running App

Give the agent a spec for a 3 tier app. It scaffolds the project, implements features, writes tests, and iterates until the app builds and runs locally end to end

App running locally + passing test suite

Autonomous Issue Fix

Pick one open issue from a repo. The agent reproduces it, locates the cause, implements a fix with a regression test, and raises a PR, with another agent reviewing the PR

Review-ready PR that closes the issue

AGENT LOOP Plan → Act (skills) → Run / test → Self-check → Human approve

BOTH TRACKS · HOW YOU'RE GRADED

Evaluation Criteria

Capstone 2 is judged as much on rigour as on the build. Evaluation has an offline arm (does it work on a known set?) and an online arm (does it stay correct in use?), scored across five weighted dimensions.

Scoring rubric

Correctness & quality	**30%**
Evaluation rigour	**25%**
Architecture & stack use	**20%**
Guardrails & governance	**15%**
Demo & communication	**10%**

Core metrics: *accuracy vs. baseline · groundedness · reconciliation pass-rate · time-to-answer · cost per query · guardrail-trigger rate.*

HEXAWARE

Thank you